

Chapter Overview

Chapter 08 covers application security in ShopVerse: JWT authentication, Spring Security filter chain, OAuth2/OIDC, password hashing, OWASP Top 10 mitigations, CORS, CSRF, rate limiting, input validation, secrets management, TLS/HTTPS, and security testing.

Sub-Chapter	Topic	Key Class / Standard
08-01	JWT – JSON Web Tokens	JwtUtil, JwtAuthFilter, HS256 vs RS256, token expiry
08-02	Spring Security Filter Chain	SecurityFilterChain, @EnableMethodSecurity, @PreAuthorize
08-03	OAuth2 & OIDC	Authorization Code flow, Resource Server, Google/GitHub SSO
08-04	Password Hashing	BCryptPasswordEncoder, Argon2, PBKDF2, cost factor
08-05	OWASP Top 10 – Injection & XSS	SQL injection, JPQL injection, XSS, CSP headers
08-06	OWASP – Broken Auth & Insecure Design	JWT blacklist, refresh token rotation, brute-force protection
08-07	CORS & CSRF	CorsConfiguration, CSRF disable for stateless, SameSite cookie
08-08	Rate Limiting & API Throttling	RateLimiterFilter, Redis sliding window, 429 Too Many Requests
08-09	Input Validation & Sanitisation	@Valid, @Pattern, HTML sanitise, file upload validation
08-10	Secrets Management	Spring Config Server, HashiCorp Vault, environment variables
08-11	TLS/HTTPS & mTLS	TLS termination, Nginx SSL, mTLS between services
08-12	Security Testing	OWASP ZAP, SAST SpotBugs, DAST, secret scanning, Trivy

JWT Structure

JWT = Base64URL(Header) . Base64URL(Payload) . Signature

Header: { "alg": "HS256", "typ": "JWT" }

Payload Claims:

sub: userId (subject)

iss: "shopverse" (issuer)

iat: 1718000000 (issued-at epoch seconds)

exp: 1718003600 (expiry = iat + 3600s)

roles: ["ROLE_CUSTOMER"] (custom claim)

Signature: HMACSHA256(base64(header)+"."+base64(payload), secret)

HS256 vs RS256

Algorithm	Key	Verify With	ShopVerse Use	Notes
HS256	Shared secret (symmetric)	Same secret	Internal service-to-service	Secret on all services
RS256	RSA key pair (asymmetric)	Public key only	External JWT (OIDC SSO)	Private key only on issuer
ES256	EC key pair (asymmetric)	Public key only	High-security	Shorter keys; same security

Token Expiry Strategy

Token	TTL	Storage	Renewal
Access Token	15 minutes	Authorization header / memory	Use refresh token to obtain new
Refresh Token	7 days	HttpOnly cookie (not JS-accessible)	Rotated on each use; old blacklisted

```
// JwtUtil.java
@Component public class JwtUtil {
    private final char[] secret; // char[] not String
    public String generate(long userId, List roles) {
        return Jwts.builder()
            .subject(String.valueOf(userId))
            .issuer("shopverse")
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + expiryMs))
            .claim("roles", roles)
            .signWith(Keys.hmacShaKeyFor(new String(secret).getBytes()), HS256)
            .compact();
    }
    public Jwts validate(String token) {
        Claims c = Jwts.parser().verifyWith(signingKey).build()
            .parseSignedClaims(token).getPayload();
        return new Jwts(Long.parseLong(c.getSubject()),
            c.get("roles", List.class));
    }
}
```

Filter Chain Order (ShopVerse)

HTTP Request enters Tomcat

1. SecurityContextPersistenceFilter – restore SecurityContext
2. CorsFilter – set CORS headers, handle preflight
3. CsrfFilter – DISABLED (stateless JWT API)
4. RateLimiterFilter (custom) – Redis sliding window -> 429 if exceeded
5. JwtAuthenticationFilter (custom) – validate JWT -> set SecurityContext
 - Extract "Authorization: Bearer <token>" header
 - JwtUtil.validate(token) -> JwtClaims
 - Set UsernamePasswordAuthenticationToken in SecurityContextHolder
6. ExceptionTranslationFilter – handle AuthenticationException/AccessDenied
7. FilterSecurityInterceptor – evaluate @PreAuthorize annotations
8. DispatcherServlet -> @RestController

```
@Configuration @EnableMethodSecurity(prePostEnabled=true)
public class SecurityConfig {
    @Bean public SecurityFilterChain chain(HttpSecurity http) throws Exception {
    return http
        .csrf(c -> c.disable())
        .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
        .cors(c -> c.configurationSource(corsSource()))
        .authorizeHttpRequests(a -> a
            .requestMatchers("/api/v1/auth/**").permitAll()
            .requestMatchers(HttpMethod.GET, "/api/v1/products/**").permitAll()
            .anyRequest().authenticated())
        .addFilterBefore(rateLimiterFilter, UsernamePasswordAuthenticationFilter.class)
        .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
        .exceptionHandling(e -> e
            .authenticationEntryPoint((req,res,ex) -> res.sendError(401))
            .accessDeniedHandler((req,res,ex) -> res.sendError(403)))
        .build();
    }
}
```

Authorization Code Flow

1. User clicks "Login with Google"

2. ShopVerse redirects to Google authorization endpoint (scope=openid email)

3. User authenticates at Google; grants consent

4. Google redirects back to /oauth2/callback?code=AUTH_CODE

5. ShopVerse exchanges code + client_secret for tokens

POST https://oauth2.googleapis.com/token -> { id_token, access_token }

6. ShopVerse validates id_token (OIDC JWT); extracts email + sub

7. ShopVerse upserts Customer record; issues own ShopVerse JWT

8. ShopVerse JWT returned to SPA client

```
# application.yml
spring.security.oauth2.client:
  registration:
    google:
      client-id: ${GOOGLE_CLIENT_ID}
      client-secret: ${GOOGLE_CLIENT_SECRET}
      scope: openid,email,profile
      provider:
        google:
          issuer-uri: https://accounts.google.com
```

OIDC Claims

Claim	Type	Description
sub	String	Subject (unique user identifier at provider)
email	String	User email address
name	String	Display name
picture	URL	Profile image URL
iat/exp	Number	Issued-at / Expiry epoch seconds

Algorithm	Type	Adaptive	Memory-Hard	ShopVerse
MD5 / SHA-1	Fast hash	No	No	Never – broken
SHA-256	Fast hash	No	No	Never – too fast for passwords
BCrypt	KDF	Yes (work factor)	No	Default (strength=12)
Argon2id	KDF	Yes	Yes	Preferred (memory=65536, iter=3)
PBKDF2	KDF	Yes (iterations)	No	Acceptable fallback

```
@Bean public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12); // ~250ms on modern hardware
}
```

```
// Usage
```

```
String hash = encoder.encode(rawPassword);
```

```
boolean ok = encoder.matches(rawPassword, storedHash);
```

```
// BCrypt output format: $2a$12$<22-char-salt><31-char-hash>
```

■ BCrypt strength=12 targets ~100-250ms per hash. Increase over time as hardware gets faster. A 250ms hash means an attacker can try only ~4 passwords/second per core offline.

Injection Prevention

Risk	Vulnerable	Safe
SQL injection	<code>createNativeQuery("SELECT * FROM @Query WHERE Email = @Email") + @Param</code>	<code>@Query("SELECT * FROM Product WHERE Email = ?")</code>
JPQL injection	<code>createQuery("FROM Product WHERE Email = @Email")</code>	<code>EntityManager.createQuery()</code> or Criteria API
Log injection	<code>log.info("User: "+input)</code>	<code>log.info("User: {}", sanitize(input))</code>
Path traversal	<code>new File(baseDir + userInput)</code>	<code>Path.of(base).resolve(input) + startsWith()</code> check

XSS Prevention

XSS Type	Attack	Prevention
Stored XSS	Malicious script saved; executed on page load	Sanitize HTML with Jsoup; Content-Security-Policy header
Reflected XSS	Script in URL reflected in response	Never reflect unsanitised input; encode output
DOM-based XSS	Client JS uses URL fragment unsafely	Framework auto-escaping (React JSX escapes by default)

```
// Jsoup HTML sanitisation
String safe = Jsoup.clean(userHtml, Safelist.basicWithImages());
// Content Security Policy header
http.headers(h -> h.contentSecurityPolicy(csp ->
csp.policyDirectives(
"default-src 'self'; script-src 'self'; img-src 'self' data: https:; object-src 'none'"))));
```

```
// JWT Blacklist for logout / token revocation
@Service public class TokenBlacklistService {
    public void blacklist(String token, long expiryMs) {
        redis.opsForValue().set("blacklist:"+token, "1", Duration.ofMillis(expiryMs));
    }
    public boolean isBlacklisted(String token) {
        return Boolean.TRUE.equals(redis.hasKey("blacklist:"+token));
    }
}
```

Brute-Force Protection Matrix

Attack	Countermeasure	Implementation
Credential stuffing	Rate limit per IP	5 login attempts/min; 429 with Retry-After
Password spraying	Account lockout	Lock after 10 fails; unlock after 30 min or email link
Token enumeration	Generic error message	Always 401 (never "wrong password" vs "user not found")
Replay attack	Short TTL + blacklist	Access token 15 min; refresh rotated + blacklisted on use
Weak password	Password policy	min 8 chars, complexity check, NIST-800-63B wordlist check

```
@Bean public CorsConfigurationSource corsSource() {  
    CorsConfiguration c = new CorsConfiguration();  
    c.setAllowedOrigins(List.of("https://shopverse.com", "https://app.shopverse.com"));  
    c.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS"));  
    c.setAllowedHeaders(List.of("Authorization", "Content-Type", "X-Request-ID"));  
    c.setAllowCredentials(true);  
    c.setMaxAge(3600L);  
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();  
    source.registerCorsConfiguration("/api/**", c);  
    return source;  
}
```

CSRF: When to Enable vs Disable

Scenario	CSRF Needed	Reason
Session cookie auth	Yes	Browser auto-sends cookie; forged cross-site request works
JWT in Authorization header	No	JS must set header explicitly; cross-origin JS blocked by CORS
JWT in HttpOnly cookie	Yes	Browser auto-sends cookie; CSRF possible even with JWT


```
@Component @Order(1)
public class RateLimiterFilter extends OncePerRequestFilter {
    @Override protected void doFilterInternal(
        HttpServletRequest req, HttpServletResponse res, FilterChain chain)
        throws IOException, ServletException {
        String key = "rate:" + req.getRemoteAddr();
        Long allowed = redis.execute(rateLimitScript,
            List.of(key), String.valueOf(MAX_REQ), String.valueOf(WINDOW_SEC),
            String.valueOf(System.currentTimeMillis()));
        if (Long.valueOf(0L).equals(allowed)) {
            res.setStatus(429);
            res.setHeader("Retry-After", "60");
            res.getWriter().write("{\"error\":\"rate_limited\"}");
            return;
        }
        chain.doFilter(req, res);
    }
}
```

Tier	Req/Min	ShopVerse Rule
Anonymous IP	60	Product browsing, search
Authenticated User	200	Cart, checkout, order history
API Key (partner)	1000	B2B integrations, catalog sync
Admin	Unlimited	Admin console operations

```
public record PlaceOrderRequest(
    @NotNull @Positive long customerId,
    @NotEmpty @Size(max=50) List<@Valid ItemRequest> items,
    @Pattern(regexp="[A-Z0-9]{6,12}", message="Invalid promo code")
    @Nullable String promoCode
) {
    public PlaceOrderRequest {
        Objects.requireNonNull(items);
        if (items.isEmpty()) throw new ValidationException("items required");
    }
}

// File upload validation
public void validateUpload(MultipartFile file) {
    if (file.getSize() > 5_000_000) throw new ValidationException("File too large (max 5MB)");
    String contentType = file.getContentType();
    Set allowed = Set.of("image/jpeg", "image/png", "image/webp");
    if (!allowed.contains(contentType)) throw new ValidationException("Invalid type: " +
        contentType);
    // Verify magic bytes (not just Content-Type header which can be spoofed)
    byte[] magic = file.getBytes();
    boolean isPng = magic[0]==0x89 && magic[1]==0x50 && magic[2]==0x4E && magic[3]==0x47;
}
```

Approach	Security	ShopVerse Use
Environment variables	Medium	DB pass, Redis pass in non-prod; injected by k8s
Spring Cloud Config (CM)	Medium	Application properties; encrypted values with jasypt
HashiCorp Vault	High	Production: DB creds, JWT secret, payment API key; dynamic rotation
Kubernetes Secrets	Medium (encrypted)	Deployment secrets mounted as env or volume
AWS Secrets Manager	High	AWS-hosted production; IAM-based access; automatic rotation

```
# application.yml - Vault integration
spring.cloud.vault:
  uri: https://vault.shopverse.internal:8200
  authentication: KUBERNETES
  kv:
    enabled: true
    backend: secret
    default-context: shopverse-api
```

■ Never commit secrets to Git – not even encrypted. Use Vault or AWS Secrets Manager. Rotate secrets automatically. Use short-lived dynamic credentials where possible.

```
# nginx.conf - TLS termination
server {
    listen 443 ssl http2;
    server_name api.shopverse.com;
    ssl_certificate /etc/letsencrypt/live/shopverse.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/shopverse.com/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-RSA-AES256-GCM-SHA384:ECDHE-RSA-CHACHA20-POLY1305;
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload";
    add_header X-Frame-Options SAMEORIGIN;
    add_header X-Content-Type-Options nosniff;
    add_header Content-Security-Policy "default-src 'self'";
    location / { proxy_pass http://shopverse-api:8080; }
}
```

Aspect	Standard TLS	mTLS
Who authenticates	Server only (presents cert to client)	Both server and client present certs
Use case	Public API	Internal microservice-to-microservice
ShopVerse	Nginx terminates TLS for public API	istio service mesh for pod-to-pod in k8s

Test Type	Tool	Finds	When
SAST (Static Analysis)	SpotBugs + find-sec-bugs	Hardcoded secrets, injection risks	CI every commit
Dependency CVEs	OWASP dependency-check, Snyk	CVEs in third-party libraries	CI every commit
DAST (Dynamic)	OWASP ZAP, Burp Suite	XSS, SQLI, auth bypass, insecure headers	Right before staging
Secret scanning	Trufflehog, GitHub secret scanner	API keys, passwords in Git history	Pre-commit hook + CI
Container scan	Trivy, Gype	CVEs in OS/app layers	Docker build step in CI
Penetration test	Manual + tools	Business logic flaws, privilege escalation	Quarterly by security team

```
# GitHub Actions security pipeline
```

```
- name: SpotBugs SAST
```

```
run: mvn com.github.spotbugs:spotbugs-maven-plugin:check
```

```
- name: OWASP Dependency Check
```

```
run: mvn org.owasp:dependency-check-maven:check -DfailBuildOnCVSS=7
```

```
- name: Trivy container scan
```

```
uses: aquasecurity/trivy-action@master
```

```
with:
```

```
image-ref: "shopverse/api:${{ github.sha }}"
```

```
severity: "HIGH,CRITICAL"
```

```
exit-code: "1"
```

■ Do

- ✓ Store JWT secrets as `char[]`; zero out on shutdown. Never as `String`.
- ✓ Use short-lived access tokens (15 min) + `HttpOnly` refresh tokens (7 days) with rotation.
- ✓ BCrypt strength=12 (>100ms per hash) to resist offline brute-force attacks.
- ✓ Set security headers: HSTS, CSP, X-Frame-Options, X-Content-Type-Options on all responses.
- ✓ Enable OWASP dependency-check in CI; fail build on CVSS >= 7.
- ✓ Use Vault or AWS Secrets Manager for production secrets – never env vars for critical creds.
- ✓ Rate limit login endpoint separately (5/min per IP) to prevent credential stuffing.
- ✓ Use `@Valid` + `@Pattern` for all user input; sanitise HTML with Jsoup for rich text fields.
- ✓ Scan Docker images with Trivy before pushing to registry; fail build on HIGH/CRITICAL CVE.

■ Anti-Patterns

- Storing JWT secret as `String` – stays in heap/pool; visible in heap dumps.
- MD5/SHA-1 for password hashing – crackable with rainbow tables in seconds.
- Wildcard CORS (`allowedOrigins("*")`) with `credentials=true` – browser rejects anyway; also a config error.
- Logging passwords or token values – even at `DEBUG` level; rotate immediately if discovered.
- Exposing stack traces in API error responses – reveals internal package structure.
- Long-lived access tokens (>1h) without refresh rotation – stolen token valid for hours.
- Secrets in `application.properties` committed to Git – even in "dev" branch.
- EAGER loading user roles/permissions – loads all data on every security check.

Sub-Chapter	Key Point	One-Liner
08-01 JWT	15min access / 7day refresh	Access short-lived; refresh rotated on each use + blacklist
08-02 Security	authFilter Before(jwtFilter,...)	JWT filter runs before UsernamePasswordAuthFilter
08-03 OAuth2	Authorization Code + PKCE	Never Implicit flow; use PKCE for SPAs
08-04 BCrypt	strength=12	~250ms per hash; adaptive cost resists brute force
08-05 Injection	Parameterised @Param always	Never concatenate user input into JPQL/SQL
08-06 Brute-force	500 attempts/min per IP	Rate limit + lockout + generic error messages
08-07 CORS	Explicit allowed origins	Never wildcard with credentials; preflight cached 3600s
08-08 Rate limit	429 + Retry-After	Sliding window in Redis via atomic Lua script
08-09 Validation	@Valid + Jsoup	Server-side validation AND HTML sanitisation for rich text
08-10 Secrets	Vault in prod	Never in Git; dynamic creds with automatic rotation
08-11 TLS	TLS 1.2/1.3 only	Disable TLS 1.0/1.1; HSTS; HTTP -> HTTPS redirect
08-12 Security	SAST + DAST + Trivy	Three-layer: static code + dynamic runtime + container

Interview Quick-Check

Question	Concise Answer
JWT vs session cookie?	JWT: stateless, scalable, revocation needs blacklist. Cookie session: server state, easy revocation
BCrypt vs Argon2?	Argon2 is recommended (memory-hard; tunable mem+time+parallelism). BCrypt widely supported.
Why disable CSRF for JWT API?	CSRF exploits cookie auto-send. JWT in Authorization header requires explicit JS; cross-origin JS
CORS preflight?	Browser OPTIONS request checks allowed origins/methods/headers. Server responds with allowed
mTLS vs TLS?	TLS: server cert only. mTLS: both client and server present certs. mTLS for zero-trust internal serv